Computed-Torque Controller

Cable-Driven 2R Manipulator — NVIDIA Isaac Sim Implementation

No Drake Dependency — Pure NumPy + PhysX ArticulationView

Isaac Sim Robotics Project

April 2026

Key Results at a Glance

- **Tracking RMS error**: $\approx 11\,\text{mm}$ on $20\,\text{mm} \times 160\,\text{mm}$ rectangle trajectory
- **Controller**: Computed torque (inverse dynamics) — **pure NumPy**, no Drake at runtime
- **Physics engine**: **NVIDIA PhysX** via Isaac Sim `ArticulationView` (GPU-acceleratable)
- **Dynamics queries**: $\boldsymbol{M}(\boldsymbol{q})$, $\boldsymbol{C}(\boldsymbol{q},\dot{\boldsymbol{q}})\dot{\boldsymbol{q}} + \boldsymbol{g}(\boldsymbol{q})$ from `ArticulationView` tensor API
- **Cable visualization**: Real-time USD cylinder prims driven by headless Drake `CablePlant`
- **Closed-loop bandwidth**: $\omega_n = \sqrt{800} \approx 28\,\text{rad/s}$, $\zeta = 0.71$ at $\Delta t = 10\,\text{ms}$
- **GPU-ready foundation**: Same controller loop compatible with Isaac Lab RL training

Contents

1 Algorithm Overview **3**
- 1.1 What the Simulation Actually Does . . . 3
- 1.2 Algorithm Flow — Step by Step . . . 3
- 1.3 What the Cable Is (and Isn't) . . . 3

2 Motivation: From PyDrake to Isaac Sim **4**

3 System Architecture **4**
- 3.1 High-Level Pipeline . . . 4
- 3.2 Import Order — Critical Rule . . . 5

4 Dynamics via PhysX ArticulationView **7**
4.1 Manipulator Equation of Motion (Review) . . . 7
4.2 PhysX Tensor API — Dynamics Queries . . . 7
4.3 PhysX Drive Configuration — Critical Fix . . . 8

5 Computed-Torque Controller — Pure NumPy **9**
5.1 Control Law . . . 9
5.2 Controller Implementation . . . 9
5.3 IK and Differential IK — NumPy Only . . . 10

6 Trajectory Generation **11**
6.1 Pure SciPy Trajectories (No Drake) . . . 11
6.2 Rectangle — Velocity Profiling . . . 11
6.3 Move-to-Start — Cubic Hermite Preamble . . . 11

7 Simulation Loop — Step by Step **12**
7.1 Setup Phase . . . 12
7.2 Control Loop . . . 12

8 Cable Visualization **14**
8.1 DrakeCablePlant — Headless Geometry Engine . . . 14

9 URDF-to-USD Pipeline **14**
9.1 Conversion Flow . . . 14
9.2 Color Mapping from URDF . . . 15

10 Torque Application **15**

11 Results and Plotting **15**
11.1 3×3 Diagnostic Plot . . . 15
11.2 Performance Metrics . . . 16

12 Running the Simulation **16**

13 Comparison: PyDrake vs Isaac Sim **18**

14 Notation Summary **18**

15 Summary **18**

16 Series Elastic Actuator (SEA) Extension **19**
16.1 Motor Rotor Dynamics (Torque Mode) . . . 19
16.2 SEA Diagnostics . . . 20

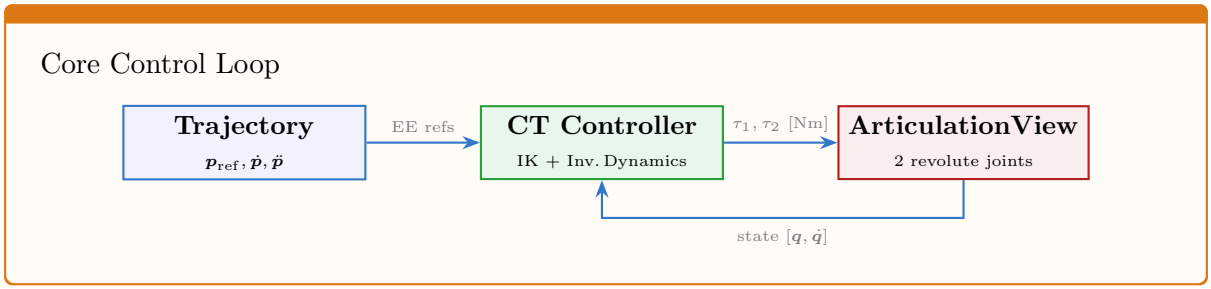
17 Residual RL Compensator **20**
17.1 Architecture . . . 20
17.2 What the Policy Learns for Weak Springs . . . 20

1 Algorithm Overview

This section summarizes what the simulation does before the detailed sections that follow.

1.1 What the Simulation Actually Does

The simulation runs a **single PhysX** `ArticulationView` loaded from a USD stage (converted from URDF) with **two revolute joints**. A pure-NumPy computed-torque controller reads joint states, computes torques via the PhysX tensor API, and writes them back via `set_joint_efforts()`. The loop runs at the PhysX default timestep (typically ∼120 Hz for rendering or faster in headless mode).



1.2 Algorithm Flow — Step by Step

Every simulation step the controller executes:

1. **Read state** $(\boldsymbol{q}, \dot{\boldsymbol{q}})$ from `ArticulationView.get_joint_positions / velocities`
2. **Evaluate trajectory** → desired EE position $\boldsymbol{p}_{\text{ref}}$, velocity $\dot{\boldsymbol{p}}_{\text{ref}}$, acceleration $\ddot{\boldsymbol{p}}_{\text{ref}}$
3. **Inverse kinematics**: $\boldsymbol{p}_{\text{ref}} \xrightarrow{\text{2R analytical}} \boldsymbol{q}_{\text{des}}$
4. **Differential IK**: $\dot{\boldsymbol{p}}_{\text{ref}} \xrightarrow{\boldsymbol{J}^{-1}} \dot{\boldsymbol{q}}_{\text{ref}}, \quad \ddot{\boldsymbol{p}}_{\text{ref}} \xrightarrow{\boldsymbol{J}^{-1}} \ddot{\boldsymbol{q}}_{\text{ref}}$
5. **PD + feedforward**: $\boldsymbol{a}_{\text{des}} = \ddot{\boldsymbol{q}}_{\text{ref}} + K_p(\boldsymbol{q}_{\text{des}} - \boldsymbol{q}) + K_d(\dot{\boldsymbol{q}}_{\text{ref}} - \dot{\boldsymbol{q}})$
6. **Inverse dynamics**: $\boldsymbol{\tau} = \boldsymbol{M}(\boldsymbol{q})\,\boldsymbol{a}_{\text{des}} + \boldsymbol{h}(\boldsymbol{q}, \dot{\boldsymbol{q}})$ via PhysX tensor API
7. **Saturate and apply**: $\boldsymbol{\tau}_{\text{clip}} = \text{clip}(\boldsymbol{\tau}, \pm\tau_{\max}) \rightarrow$ `set_joint_efforts()`

1.3 What the Cable Is (and Isn't)

The cable/belt is NOT modeled as a physical element. The PhysX articulation contains only two rigid revolute joints—there is no spring, cable, or pulley in the physics simulation. The belt is treated as a **rigid tendon**: an inextensible, massless transmission between motor and joint.

The controller writes **joint torques** $[\tau_1, \tau_2]$ in Nm directly via `set_joint_efforts()`. There is no cable compliance, no pulley friction, and no belt elasticity in the dynamics.

Cable tensions are a **post-hoc decomposition** for logging:

$$F_{\text{net}} = \frac{\tau_2}{r_p}, \qquad T_{\text{green}} = \max(F_{\text{net}}, 0), \qquad T_{\text{red}} = \max(-F_{\text{net}}, 0)$$

These appear in the log arrays and plots but are never fed back into PhysX. The cable visualization (coloured polylines in the Isaac Sim viewport) is likewise computed from FK for rendering only.

2 Motivation: From PyDrake to Isaac Sim

The PyDrake implementation (see companion document) validates the computed-torque controller with analytical dynamics and Meshcat visualization. **This Isaac Sim version** re-implements the same control law using NVIDIA's physics stack for two reasons:

1. **GPU acceleration**: PhysX runs natively on CUDA, enabling future RL training with thousands of parallel environments.
2. **Isaac Lab compatibility**: The same `ArticulationView` API used here is used by IsaacLab for GPU-parallel simulation.

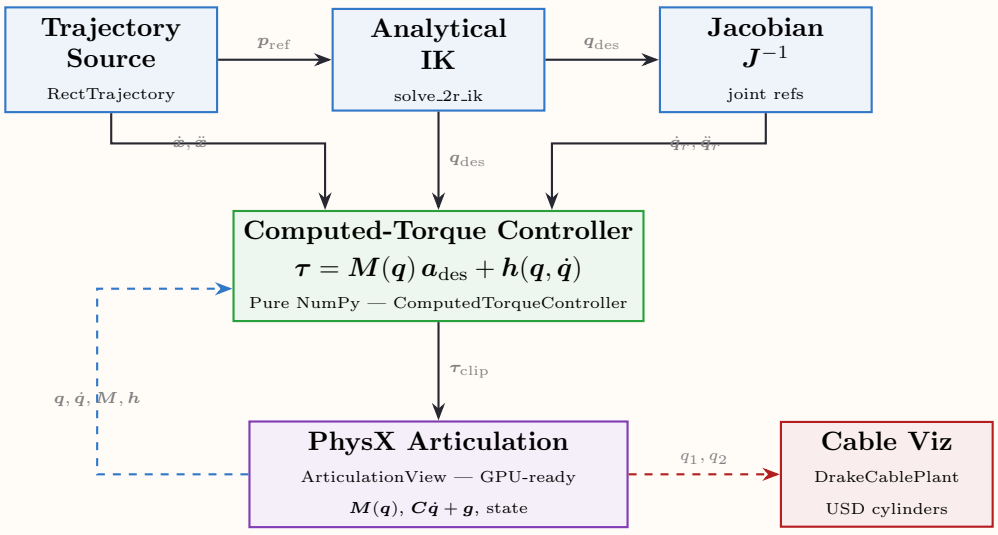
Key Architectural Difference

Component	PyDrake Version	Isaac Sim Version
Physics engine	Drake MultibodyPlant	NVIDIA PhysX 5
$\boldsymbol{M}(\boldsymbol{q})$ source	`CalcMassMatrix()`	`ArticulationView` tensor
$\boldsymbol{h}(\boldsymbol{q}, \dot{\boldsymbol{q}})$ source	`CalcBiasTerm()`	`get_coriolis_and_centrifugal()` + `get_generalized_gravity()`
Controller	Drake `LeafSystem`	Pure NumPy function
Visualization	Meshcat (browser)	Isaac Sim native viewport / USD
IK solver	Drake `InverseKinematics`	Pure NumPy 2R closed-form
Trajectory	`PiecewisePolynomial`	`scipy.CubicSpline`

3 System Architecture

3.1 High-Level Pipeline

Isaac Sim CT Controller Pipeline



3.2 Import Order — Critical Rule

Isaac Sim requires `SimulationApp` to be instantiated **before any other** `omni.*` **or** `isaacsim.*` **import**. Violating this crashes Python without a useful error message.

Quiet startup via `IsaacSimLogger`

Isaac Sim writes ∼500 lines of kit/carb noise (extension loading, GPU bandwidth tables, deprecation warnings) to file descriptors 1 and 2 **from C++**, bypassing `sys.stdout`. The `IsaacSimLogger` uses `os.dup2` to redirect fds at the OS level:

```python
class IsaacSimLogger:
    @classmethod
    def from_argv(cls) -> "IsaacSimLogger":
        """Verbose mode if --verbose in sys.argv, else quiet."""
        return cls(verbose=("--verbose" in sys.argv))

    def suppress(self):
        """Redirect fds 1+2 to /dev/null (OS-level, bypasses C++)."""
        if self._verbose: return
        self._saved1 = os.dup(1);  self._saved2 = os.dup(2)
        devnull = os.open(os.devnull, os.O_WRONLY)
        os.dup2(devnull, 1);  os.dup2(devnull, 2)
        os.environ["CARB_LOG_LEVEL"] = "error"
        atexit.register(self._restore_fds)  # crash safety

    def restore(self):
        """Restore fds; print Isaac Sim ready message."""
        if self._verbose: return
        self._restore_fds()
        elapsed = time.time() - self._t0
        print(f"Isaac Sim ready ({elapsed:.1f}s) --- use --verbose
            for full log")
```

Listing 1: `project_utils/log_isaacsim.py` — key methods

502 lines before / 139 lines after (72% reduction). Normal Python tracebacks remain fully visible because fds are restored before the custom `excepthook` fires.

```python
import os, sys
from pathlib import Path

# Project root on sys.path (needed before SimulationApp)
_PROJECT_ROOT = str(Path(__file__).resolve().parent)
if _PROJECT_ROOT not in sys.path:
    sys.path.insert(0, _PROJECT_ROOT)

# Pre-parse --render (cannot use argparse yet)
_RENDER_CHOICES = ("native", "websocket", "headless")
_render_mode = "native"
for _i, _arg in enumerate(sys.argv):
    if _arg == "--render" and _i + 1 < len(sys.argv):
```

```
        _render_mode = sys.argv[_i + 1]
        if _render_mode not in _RENDER_CHOICES:
            print(f"[ERROR] --render must be one of {_RENDER_CHOICES}")
            sys.exit(1)

# QUIET mode: suppress Isaac Sim extension/GPU/warning noise at startup.
# Use --verbose to restore original output.
from project_utils.log_isaacsim import IsaacSimLogger
_log = IsaacSimLogger.from_argv()  # no-op when --verbose in sys.argv
_log.suppress()

import warnings
warnings.filterwarnings("ignore", category=DeprecationWarning)

# FIRST Isaac Sim import
from isaacsim import SimulationApp
simulation_app = SimulationApp({
    "headless": _render_mode != "native",  # native -> local window
    "width": 1280, "height": 720,
    "hide_ui": True,
})

# NOW safe to import Isaac Sim modules
from omni.isaac.core import World
from omni.isaac.core.articulations import ArticulationView

# WebRTC streaming (--render websocket) -- set up BEFORE world init
if _render_mode == "websocket":
    import subprocess
    from omni.kit.livestream.webrtc import set_setting, enable_extension
    try:
        _ip = subprocess.check_output(
            ["tailscale", "ip", "-4"], text=True).strip()
    except Exception:
        _ip = "127.0.0.1"  # fallback: localhost
    set_setting("/app/livestream/publicEndpointAddress", _ip)
    enable_extension("omni.kit.livestream.webrtc")
    _log.print(f"  WebRTC (Tailscale) : connect to  {_ip} : 49100")

# Startup complete -- restore stdout/stderr
_log.restore()
```

Listing 2: Mandatory import sequence — `script_cup_manipulator_pendulam_tendon_computed_torque_isaac_sim.py`

WebRTC streaming to Mac / tablet via Tailscale:

1. Install Tailscale on both machines and join the same tailnet.
2. Start with `--render websocket`.
3. Open the Isaac Sim WebRTC Streaming Client on the Mac; connect to the printed Tailscale IP on port 49100.

Critical: `set_setting("/app/livestream/publicEndpointAddress", ip)` must point to the Tailscale IP, not the LAN IP, for ICE negotiation to succeed across the VPN. Without it the Mac client receives an unreachable address and the stream never opens.

4 Dynamics via PhysX ArticulationView

4.1 Manipulator Equation of Motion (Review)

The same rigid-body dynamics apply regardless of the physics engine:

$$\boxed{\boldsymbol{M}(\boldsymbol{q})\,\ddot{\boldsymbol{q}} + \underbrace{\boldsymbol{C}(\boldsymbol{q},\dot{\boldsymbol{q}})\,\dot{\boldsymbol{q}} + \boldsymbol{g}(\boldsymbol{q})}_{\boldsymbol{h}(\boldsymbol{q},\dot{\boldsymbol{q}})} = \boldsymbol{\tau}} \tag{1}$$

where:

- $\boldsymbol{M}(\boldsymbol{q}) \in \mathbb{R}^{2\times 2}$ — mass (inertia) matrix
- $\boldsymbol{C}(\boldsymbol{q},\dot{\boldsymbol{q}})\,\dot{\boldsymbol{q}} \in \mathbb{R}^2$ — Coriolis and centrifugal forces
- $\boldsymbol{g}(\boldsymbol{q}) \in \mathbb{R}^2$ — gravitational torques
- $\boldsymbol{h}(\boldsymbol{q},\dot{\boldsymbol{q}}) = \boldsymbol{C}\dot{\boldsymbol{q}} + \boldsymbol{g}$ — bias vector

4.2 PhysX Tensor API — Dynamics Queries

Isaac Sim exposes these quantities through the `ArticulationView` in a GPU-tensor-compatible format:

```python
class CupManipulatorTendonIsaac:
    def initialize_dynamics_view(self, world):
        """Create ArticulationView for dynamics (M, C, g) queries."""
        self._art_view = ArticulationView(
            prim_paths_expr="/World/manipulator_cable",
            name="manip_dynamics",
        )
        world.scene.add(self._art_view)
        self._art_view.initialize(world.physics_sim_view)
```

Listing 3: Dynamics initialization — `robots/cup_manipulator_tendon_isaac.py`

```python
def get_mass_matrix(self):
    """Return (2,2) mass matrix for actuated DOFs."""
    M_full = self._art_view.get_mass_matrices()  # (1, n_dof, n_dof)
    M = M_full[0]  # single environment
    idx = self._actuated_dof_indices
    return M[np.ix_(idx, idx)]  # extract actuated 2x2 block

def get_coriolis_and_centrifugal(self):
    """Return (2,) Coriolis+centrifugal forces."""
    return self._art_view.get_coriolis_and_centrifugal_forces()[0][
        self._actuated_dof_indices]

def get_generalized_gravity(self):
    """Return (2,) gravity torques."""
    return self._art_view.get_generalized_gravity_forces()[0][
        self._actuated_dof_indices]
```

```python

def get_bias_forces(self):
    """h(q, q_dot) = C(q,q_dot)q_dot + g(q)"""
    return self.get_coriolis_and_centrifugal() \
         + self.get_generalized_gravity()
```

Listing 4: Mass matrix and bias forces — `robots/cup_manipulator_tendon_isaac.py`

▶ **Equations implemented by the above code**

The three PhysX tensor calls map directly to the terms in (1):

`get_mass_matrices()`	$\rightarrow$	$\boldsymbol{M}(\boldsymbol{q}) \in \mathbb{R}^{2\times 2}$
`get_coriolis_and_centrifugal_forces()`	$\rightarrow$	$\boldsymbol{C}(\boldsymbol{q},\dot{\boldsymbol{q}})\,\dot{\boldsymbol{q}} \in \mathbb{R}^2$
`get_generalized_gravity_forces()`	$\rightarrow$	$\boldsymbol{g}(\boldsymbol{q}) \in \mathbb{R}^2$

The bias vector is their sum: $\boldsymbol{h} = \boldsymbol{C}\dot{\boldsymbol{q}} + \boldsymbol{g}$.
Index mapping: The URDF may contain passive/fixed joints. The `_actuated_dof_indices` array selects only the two revolute DOFs $[q_1, q_2]$ from the full articulation state.

4.3 PhysX Drive Configuration — Critical Fix

PhysX Drive Stiffness Bug — Root Cause of Initial Failure When loading a URDF, Isaac Sim maps the `<limit effort="10"/>` attribute to the PhysX `angular drive`'s `maxForce=10`. Furthermore, PhysX drives default to a non-zero `stiffness`, creating a **position spring** that fights the effort commands.
For pure effort (torque) control, both must be overridden:

$$\texttt{stiffness} = 0, \qquad \texttt{maxForce} = 10^4 \text{ Nm}$$

Otherwise the 5.3 kg arm's gravity (≈ 10 Nm) consumes the entire effort budget.

```python
def add_joint_actuators(self):
    """Configure PhysX angular drives for pure effort control."""
    for jt_name in [self.JT1_NAME, self.JT2_NAME]:
        drive = UsdPhysics.DriveAPI.Get(joint_prim, "angular")
        # CRITICAL: zero stiffness (no position spring)
        drive.GetStiffnessAttr().Set(0.0)
        # CRITICAL: high maxForce (override URDF effort="10")
        drive.GetMaxForceAttr().Set(1e4)
        # Damping from config (viscous friction model)
        drive.GetDampingAttr().Set(float(jt_cfg.damping))
```

Listing 5: PhysX drive fix — `robots/cup_manipulator_tendon_isaac.py`

▶ **Equations implemented by the above code**

PhysX applies the drive force as:

$$\tau_{\text{PhysX}} = \underbrace{k_s \cdot (q_{\text{target}} - q)}_{\text{spring (must be 0)}} + \underbrace{k_d \cdot (\dot{q}_{\text{target}} - \dot{q})}_{\text{damping}} + \underbrace{\tau_{\text{effort}}}_{\text{our command}}$$

Clamped to $[-\texttt{maxForce},\ +\texttt{maxForce}]$. Setting $k_s = 0$ and $\texttt{maxForce} = 10^4$ ensures our

computed-torque command $\boldsymbol{\tau}$ passes through to the articulation without interference.

5 Computed-Torque Controller — Pure NumPy

5.1 Control Law

Identical to the PyDrake version (Section 2 below), but using NumPy arrays instead of Drake `LeafSystem` callbacks:

$$\boxed{\boldsymbol{a}_{\text{des}} = \ddot{\boldsymbol{q}}_{\text{ref}} + K_p\,(\boldsymbol{q}_{\text{des}} - \boldsymbol{q}) + K_d\,(\dot{\boldsymbol{q}}_{\text{ref}} - \dot{\boldsymbol{q}})} \tag{2}$$

$$\boxed{\boldsymbol{\tau} = \boldsymbol{M}(\boldsymbol{q})\,\boldsymbol{a}_{\text{des}} + \boldsymbol{h}(\boldsymbol{q}, \dot{\boldsymbol{q}})} \tag{3}$$

The closed-loop error dynamics are:

$$\ddot{\boldsymbol{e}} + K_d\,\dot{\boldsymbol{e}} + K_p\,\boldsymbol{e} = \boldsymbol{0} \qquad \Longrightarrow \qquad \omega_n = \sqrt{K_p},\ \zeta = \frac{K_d}{2\sqrt{K_p}} \tag{4}$$

For $K_p = 800$, $K_d = 40$: $\omega_n \approx 28.3\,\text{rad/s}$, $\zeta \approx 0.71$ (underdamped). At $\Delta t = 10\,\text{ms}$ (100 Hz), the Nyquist frequency is $\pi/\Delta t \approx 314\,\text{rad/s}$, giving a ratio $\omega_n/\omega_{\text{Nyq}} \approx 0.09$ — well within the safe sampling margin.

5.2 Controller Implementation

```python
class ComputedTorqueController:
    def __init__(self, Kp=800.0, Kd=40.0, tau_max=50.0,
                 pulley_radius=60*0.005/(2*np.pi)):
        self.Kp = float(Kp)
        self.Kd = float(Kd)
        self.tau_max = float(tau_max)
        self.r_p = float(pulley_radius)

    def compute(self, q, q_dot, q_des, q_dot_ref, q_ddot_ref, M, h):
        # PD + feedforward acceleration
        a_des = q_ddot_ref + self.Kp * (q_des - q) \
                           + self.Kd * (q_dot_ref - q_dot)
        # Inverse dynamics
        tau_raw = M @ a_des + h
        # Cable tension decomposition (joint 2)
        F_net = tau_raw[1] / self.r_p
        T_green = max(F_net, 0.0)
        T_red   = max(-F_net, 0.0)
        tau2_cmd = (T_green - T_red) * self.r_p
        # Clip
        tau_clip = np.clip(
            np.array([tau_raw[0], tau2_cmd]),
            -self.tau_max, self.tau_max)
        return CTControllerOutput(tau_clip=tau_clip, ...)
```

Listing 6: Engine-agnostic CT controller — `controller/computed_torque_isaacsim.py`

▶ **Equations implemented by the above code**

The controller is **identical** to the PyDrake version — only the *source* of $\boldsymbol{M}$ and $\boldsymbol{h}$ changes:

- **PyDrake**: `plant.CalcMassMatrix(context)`, `plant.CalcBiasTerm(context)`
- **Isaac Sim**: `art_view.get_mass_matrices()`, `get_coriolis+gravity`

Cable tension decomposition:

$$F_{\text{net}} = \frac{\tau_2}{r_{\text{p}}}, \quad T_{\text{green}} = \max(F_{\text{net}}, 0), \quad T_{\text{red}} = \max(-F_{\text{net}}, 0)$$

$$\tau_{2,\text{cmd}} = (T_{\text{green}} - T_{\text{red}}) \cdot r_{\text{p}}, \qquad \boldsymbol{\tau}_{\text{clip}} = \text{clip}(\boldsymbol{\tau}, -\tau_{\max}, +\tau_{\max})$$

5.3 IK and Differential IK — NumPy Only

```python
def ik_to_joint_space_references(
    ee_pos_ref, ee_vel_ref, ee_acc_ref,
    L1, L2, q_seed, solve_ik_fn):
    """EE refs -> joint-space (q_des, q_dot_ref, q_ddot_ref)."""
    q_des, ok = solve_ik_fn(L1, L2, ee_pos_ref, q_seed)
    # Analytical Jacobian at q_des
    q1d, q2d = q_des
    s1, c1 = np.sin(q1d), np.cos(q1d)
    s12, c12 = np.sin(q1d+q2d), np.cos(q1d+q2d)
    J = np.array([
        [-L1*s1 - L2*s12, -L2*s12],
        [ L1*c1 + L2*c12,  L2*c12]])
    J_inv = np.linalg.pinv(J)
    q_dot_ref  = J_inv @ ee_vel_ref
    q_ddot_ref = J_inv @ ee_acc_ref
    return q_des, q_dot_ref, q_ddot_ref, ok
```

Listing 7: IK-to-joint-space pipeline — `controller/computed_torque_isaacsim.py`

▶ **Equations implemented by the above code**

Step 1 — Position IK (closed-form 2R):

$$\cos q_2 = \frac{x^2 + y^2 - L_1^2 - L_2^2}{2L_1L_2}, \quad q_1 = \text{atan2}(y, x) - \text{atan2}(L_2 s_2,\ L_1 + L_2 c_2)$$

Step 2 — Velocity mapping via Jacobian:

$$\dot{\boldsymbol{q}}_{\text{ref}} = \boldsymbol{J}^{-1}(\boldsymbol{q}_{\text{des}})\, \dot{\boldsymbol{x}}_{\text{ref}}$$

Step 3 — Acceleration mapping (simplified, drops $\dot{\boldsymbol{J}}\dot{\boldsymbol{q}}$ bias):

$$\ddot{\boldsymbol{q}}_{\text{ref}} \approx \boldsymbol{J}^{-1}(\boldsymbol{q}_{\text{des}})\, \ddot{\boldsymbol{x}}_{\text{ref}}$$

6 Trajectory Generation

6.1 Pure SciPy Trajectories (No Drake)

The Isaac Sim version uses `scipy.interpolate.CubicSpline` with periodic boundary conditions to create C^2 trajectories:

```
class LoopingCubicTrajectory:
    def __init__(self, t_breaks, waypoints, period):
        self.period = period
        self._cs = CubicSpline(t_breaks, waypoints, bc_type='periodic
            ')
        self._cs_vel = self._cs.derivative(1)
        self._cs_acc = self._cs.derivative(2)

    def eval_position(self, t):
        return self._cs(t % self.period)

    def eval_velocity(self, t):
        return self._cs_vel(t % self.period)

    def eval_acceleration(self, t):
        return self._cs_acc(t % self.period)
```

Listing 8: Looping cubic spline — `controller/trajectory.py`

▶ **Equations implemented by the above code**

Periodic cubic spline satisfies at the wrap boundary:

$$\boldsymbol{p}(0) = \boldsymbol{p}(T), \quad \dot{\boldsymbol{p}}(0) = \dot{\boldsymbol{p}}(T), \quad \ddot{\boldsymbol{p}}(0) = \ddot{\boldsymbol{p}}(T) \qquad (C^2 \text{ continuity})$$

The time modulus $t \mapsto t \bmod T$ produces an infinite looping trajectory from a single lap.

6.2 Rectangle — Velocity Profiling

The rectangle uses **non-uniform time stamps** for velocity profiling (slow at corners, fast on straights):

$$v(s) = v_{\text{corner}} + (v_{\max} - v_{\text{corner}}) \cdot h\left(\frac{d(s)}{d_{\text{blend}}}\right), \qquad h(t) = t^2(3 - 2t) \tag{5}$$

where $d(s)$ is the arc-length distance to the nearest corner and $h(t)$ is the Hermite smoothstep interpolant.

6.3 Move-to-Start — Cubic Hermite Preamble

Before tracking begins, a **cubic Hermite spline** moves the arm from rest to the first waypoint:

```
class PreambleTrajectorySource:
    """During t < move_duration: approach spline.
       After: main looping trajectory."""
    def eval_position(self, t):
        if self._move is not None and t < self.move_duration:
            return self._move.eval_position(t)
        return self._main.eval_position(t - self.move_duration)
```

Listing 9: Preamble construction — `controller/trajectory.py`

Boundary conditions:

$$\boldsymbol{p}(0) = \boldsymbol{p}_{\text{home}}, \quad \dot{\boldsymbol{p}}(0) = \mathbf{0}, \quad \boldsymbol{p}(T_m) = \boldsymbol{p}_{\text{traj}}(0), \quad \dot{\boldsymbol{p}}(T_m) = \dot{\boldsymbol{p}}_{\text{traj}}(0) \tag{6}$$

7 Simulation Loop — Step by Step

7.1 Setup Phase

```
# 1. Create robot wrapper
manip = CupManipulatorTendonIsaac(MANIP_CONFIG)
manip.prepare_usd()            # Convert URDF -> USD

# 2. Create World
world = World(
    stage_units_in_meters=1.0,
    physics_dt=args.dt,        # 10 ms
    rendering_dt=args.dt,
)
world.scene.add_default_ground_plane()

# 3. Load robot
manip.load_urdf(world)
manip.weld_base_to_world(position=np.zeros(3), orientation=orient)
manip.add_end_effector_frame()
manip.set_joint_properties()
manip.add_joint_actuators()    # stiffness=0, maxForce=1e4

# 4. Cable viz -- prims BEFORE world.reset()
cable_viz = CableVisualizerIsaac(stage, drake_urdf)
cable_viz.create_prims(q1=q1_init, q2=q2_init)

# 5. Reset and initialize
world.reset()
manip.initialize_state()
manip.initialize_dynamics_view(world)
```

Listing 10: World and robot setup — `script_cup_manipulator_pendulam_tendon_computed_torque_isaac_sim.py`

> The cable USD cylinder prims must be created **before** `world.reset()` to avoid invalidating the `ArticulationView`. Adding/removing prims after reset triggers a PhysX topology change that crashes `physics_sim_view`.

7.2 Control Loop

```
while not stop_requested and step < max_steps:
    # 1. Read current state from PhysX
    q     = manip.get_positions_user_order()     # [q1, q2]
    q_dot = manip.get_velocities_user_order()    # [dq1, dq2]

    # 2. Evaluate trajectory at current time
    ee_pos_ref = traj_source.eval_position(t)
    ee_vel_ref = traj_source.eval_velocity(t)
```

```
    ee_acc_ref = traj_source.eval_acceleration(t)

    # 3. IK + Jacobian -> joint-space references
    q_des, q_dot_ref, q_ddot_ref, ik_ok = \
        ik_to_joint_space_references(
            ee_pos_ref, ee_vel_ref, ee_acc_ref,
            L1, L2, last_q_des, solve_2r_ik)
    if ik_ok:
        last_q_des = q_des.copy()      # warm-start seed

    # 4. Dynamics queries from PhysX
    M = manip.get_mass_matrix()         # (2,2)
    h = manip.get_bias_forces()         # (2,)

    # 5. Computed torque
    ct_out = ct.compute(q, q_dot, q_des, q_dot_ref,
                        q_ddot_ref, M, h)

    # 6. Apply torques to articulation
    manip.set_joint_torques(ct_out.tau_clip)

    # 7. Update cable visualization (every N steps)
    if step % _CABLE_UPDATE_INTERVAL == 0:
        cable_viz.update(q[0], q[1])

    # 8. Step physics
    # render=True for native window and websocket stream; False only
       for headless
    world.step(render=(_render_mode != "headless"))
    t += args.dt;  step += 1
```

Listing 11: Main simulation loop.

WebRTC streaming requires `render=True`. Setting `render=False` skips the viewport update, so the Mac client receives no frames. The correct predicate is `render=(_render_mode != "headless")` — not `== "native"`.

▶ Equations implemented by the above code

Each iteration of the loop executes the full CT pipeline:
(1) State read: $[\boldsymbol{q}, \dot{\boldsymbol{q}}] \leftarrow$ `ArticulationView`
(2) Reference: $[\boldsymbol{p}_r(t), \dot{\boldsymbol{p}}_r(t), \ddot{\boldsymbol{p}}_r(t)] \leftarrow$ cubic spline
(3) IK: $\boldsymbol{p}_r \xrightarrow{\text{2R}} \boldsymbol{q}_d, \quad \dot{\boldsymbol{p}}_r \xrightarrow{\boldsymbol{J}^{-1}} \dot{\boldsymbol{q}}_r, \quad \ddot{\boldsymbol{p}}_r \xrightarrow{\boldsymbol{J}^{-1}} \ddot{\boldsymbol{q}}_r$
(4) Dynamics: $\boldsymbol{M}(\boldsymbol{q}), \boldsymbol{h}(\boldsymbol{q}, \dot{\boldsymbol{q}}) \leftarrow$ PhysX tensors
(5) Control: $\boldsymbol{a}_d = \ddot{\boldsymbol{q}}_r + K_p(\boldsymbol{q}_d - \boldsymbol{q}) + K_d(\dot{\boldsymbol{q}}_r - \dot{\boldsymbol{q}}), \quad \boldsymbol{\tau} = \boldsymbol{M}\,\boldsymbol{a}_d + \boldsymbol{h}$
(6) Apply: $\boldsymbol{\tau}_{\text{clip}} \rightarrow$ PhysX drive effort
(7) Cable viz: Update USD cylinder transforms (every 5 steps for performance)
(8) Physics step: PhysX integrates one Δt

8 Cable Visualization

8.1 DrakeCablePlant — Headless Geometry Engine

Cable routing geometry cannot be computed purely from PhysX (it has no cable model). Instead, we use a **headless Drake plant** solely for cable FK:

```python
class CableVisualizerIsaac:
    def __init__(self, stage, drake_urdf):
        self._stage = stage
        self._drake = DrakeCablePlant(drake_urdf, 0.0, 0.0)
        self._prim_paths = {}

    def create_prims(self, q1, q2):
        """Pre-allocate USD cylinders at initial pose."""
        self._drake.update(q1, q2)
        self._draw_cables_usd(self._stage, self._drake)

    def update(self, q1, q2):
        """Recompute cable routing and update USD transforms."""
        self._drake.update(q1, q2)
        # Update existing prim positions (no create/delete)
        for route, pts in self._drake.get_cable_world_points():
            for i in range(len(pts) - 1):
                _update_cylinder_prim(
                    self._prim_paths[key],
                    pts[i], pts[i+1])
```

Listing 12: Cable visualization wrapper — `project_utils/viz_cables_isaacsim.py`

> This cable visualization is **NOT suitable for parallel RL training** — it runs a CPU-bound Drake plant and modifies the USD stage directly. For RL with > 1 environment, cable visualization should be replaced with a GPU-native approach or disabled entirely.

9 URDF-to-USD Pipeline

9.1 Conversion Flow

Isaac Sim requires USD format. The URDF must be converted:


```python
def prepare_usd(self):
    """Convert URDF -> USD (BEFORE World creation)."""
    import omni.kit.commands
    result, prim_path = omni.kit.commands.execute(
        "URDFParseAndImportFile",
        urdf_path=self.config.urdf_path,
        dest_path="/World/manipulator_cable",
    )

def load_urdf(self, world):
    """After USD is on stage, configure articulation."""
```

```
12      self.apply_urdf_colors()   # Map <material> -> USD
```

Listing 13: URDF to USD conversion — `robots/cup_manipulator_tendon_isaac.py`

9.2 Color Mapping from URDF

The URDF `<material><color>` tags are translated to USD material sublayers via `apply_urdf_colors()`, preserving the visual appearance from the CAD model.

10 Torque Application

```
def set_joint_torques(self, torques):
    """Apply [tau1, tau2] to articulation via ArticulationView."""
    effort = np.zeros(self._art_view.num_dof)
    for i, idx in enumerate(self._actuated_dof_indices):
        effort[idx] = float(torques[i])
    self._art_view.set_joint_efforts(
        effort.reshape(1, -1))  # (1, n_dof) for 1 env
```

Listing 14: Applying torques to PhysX — `robots/cup_manipulator_tendon_isaac.py`

▶ Equations implemented by the above code

The effort is injected into the PhysX solver as a generalized force. With the drive configured as:

$$k_s = 0 \text{ (no spring)}, \qquad \texttt{maxForce} = 10^4 \text{ Nm}$$

the effective torque on joint i is:

$$\tau_i^{\text{PhysX}} = \text{clip}\Big(\tau_i^{\text{cmd}} + k_d\,(\dot{q}_{\text{target}} - \dot{q}_i),\ -\tau_{\max}^{\text{PhysX}},\ +\tau_{\max}^{\text{PhysX}}\Big)$$

With k_d set to a small viscous damping (0.05 Nm·s/rad), the commanded torque dominates.

11 Results and Plotting

11.1 3×3 Diagnostic Plot

The `plot_results()` function generates the same diagnostic layout as the PyDrake version:

Table 1: Plot layout.

	Position	Velocity	Acceleration
EE	x, y vs ref	$\dot{x}, \dot{y}$ via $\boldsymbol{J\dot{q}}$	$\ddot{x}, \ddot{y}$
Joints	q_1, q_2 [deg]	$\dot{q}_1, \dot{q}_2$	$\ddot{q}_1, \ddot{q}_2$
Forces	τ req vs app	Cable tensions	EE XY path

Derived signals:

- EE velocity actual: $\dot{\boldsymbol{p}} = \boldsymbol{J}(\boldsymbol{q})\,\dot{\boldsymbol{q}}$ (exact, no finite differences)
- EE acceleration actual: $\ddot{\boldsymbol{p}} = \texttt{np.gradient}(\dot{\boldsymbol{p}}, t)$

- Joint-space references: $\dot{\boldsymbol{q}}_r = \boldsymbol{J}^{-1}\,\dot{\boldsymbol{x}}_r$, $\ddot{\boldsymbol{q}}_r = \boldsymbol{J}^{-1}\,\ddot{\boldsymbol{x}}_r$

```
# EE actual via FK (no Drake)
ee_x = np.array([forward_kinematics_2r(L1, L2, q[k,0], q[k,1])[0]
                 for k in range(len(t))])
ee_y = np.array([forward_kinematics_2r(L1, L2, q[k,0], q[k,1])[1]
                 for k in range(len(t))])

# EE velocity via Jacobian (analytical, no finite diff)
for k in range(len(t)):
    J = analytical_jacobian_2r(L1, L2, q[k,0], q[k,1])
    v = J @ q_dot[k]
    ee_vx[k], ee_vy[k] = v[0], v[1]
```

Listing 15: FK-based EE actual computation — `plot_results()`

Insert Isaac Sim CT results plot here.

`plots/ct_isaac_rect_20260409_*.png`

Figure 1: Isaac Sim computed-torque tracking results (rectangle). $K_p = 800$, $K_d = 40$, $\tau_{\max} = 50\,\text{Nm}$, $\Delta t = 10\,\text{ms}$.

11.2 Performance Metrics

EE tracking RMS	$\approx 11\,\text{mm}$
Laps completed	4 (in 43 s wall time for 40 s sim time)
Peak τ_1	$\approx 15\,\text{Nm}$ (within 50 Nm budget)
Physics dt	10 ms (100 Hz)
Wall-clock RTF	$\sim 1\times$ real-time (including render)
GPU	NVIDIA RTX 5090 (31 GiB)

12 Running the Simulation

```
# Activate Isaac Sim conda environment
conda activate env_isaacsim

# Default: quiet mode, computed-torque, rectangle trajectory, native
    window
python
    script_cup_manipulator_pendulam_tendon_computed_torque_isaac_sim.
    py
```

```
# Custom gains and circle trajectory, headless (benchmarking / RL)
python script_cup_manipulator_pendulam_tendon_computed_torque_isaac_sim.py \
    --render headless \
    --ct-kp 800 --ct-kd 40 --ct-tau-max 50 \
    --traj-shape circle --traj-cx 0.4 --traj-radius 0.08 \
    --duration 15

# WebRTC streaming to Mac (Tailscale-routed, port 49100)
python script_cup_manipulator_pendulam_tendon_computed_torque_isaac_sim.py \
    --render websocket
# -> outputs: Mac client: connect to  100.94.173.113 : 49100

# Verbose: restore full Isaac Sim startup log
python script_cup_manipulator_pendulam_tendon_computed_torque_isaac_sim.py \
    --render headless --verbose

# Scene visualization only (no control)
python script_cup_manipulator_pendulam_tendon_scene_viz_isaac_sim.py \
    --render websocket
```

Render mode summary

–render	Display	Use case
`native`	Local OS window	Development on the GPU machine
`websocket`	WebRTC stream, port 49100	Remote view from Mac / tablet
`headless`	None	Benchmarking, RL training, CI

13 Comparison: PyDrake vs Isaac Sim

Table 2: Side-by-side comparison of both implementations.

Aspect	PyDrake	Isaac Sim
Physics engine	Drake MultibodyPlant (CPU)	NVIDIA PhysX 5 (GPU-capable)
Controller	Drake `LeafSystem`	Pure NumPy class
Signal routing	Drake `DiagramBuilder`	Manual loop variable passing
Mass matrix	`CalcMassMatrix(ctx)`	`art_view.get_mass_matrices()`
Bias forces	`CalcBiasTerm(ctx)`	`get_coriolis()` + `get_gravity()`
IK solver	Both: closed-form 2R	Both: closed-form 2R
Trajectory	`PiecewisePolynomial`	`scipy.CubicSpline`
Visualization	Meshcat (browser)	Native viewport or headless
Cable viz	Meshcat polylines	USD cylinder prims
Timestep	2 ms (500 Hz)	10 ms (100 Hz)
RL ready?	No (CPU-only)	Yes (ArticulationView is GPU-parallel)

14 Notation Summary

Table 3: Notation reference.

Symbol	Description	Unit
$\boldsymbol{q} = [q_1, q_2]^\top$	Joint positions	rad
$\dot{\boldsymbol{q}}$	Joint velocities	rad/s
$\ddot{\boldsymbol{q}}$	Joint accelerations	rad/s^2
$\boldsymbol{M}(\boldsymbol{q})$	Mass matrix	kg·m^2
$\boldsymbol{h}(\boldsymbol{q}, \dot{\boldsymbol{q}})$	Bias vector $= \boldsymbol{C}\dot{\boldsymbol{q}} + \boldsymbol{g}$	Nm
$\boldsymbol{\tau}$	Applied joint torques	Nm
$\boldsymbol{J}(\boldsymbol{q})$	Analytical Jacobian	m/rad
$\boldsymbol{p}_{\text{EE}}$	EE position $[x, y]$	m
K_p, K_d	PD gains	s^{-2}, s^{-1}
ω_n, ζ	Natural frequency, damping ratio	rad/s, —
r_{p}	Pulley pitch radius	m
$T_{\text{green}}, T_{\text{red}}$	Cable tensions	N
L_1, L_2	Link lengths	m
Δt	Physics timestep	s

15 Summary

Implementation Summary

1. **No Drake at runtime**: Isaac Sim's `ArticulationView` provides $\boldsymbol{M}(\boldsymbol{q})$ and $\boldsymbol{h}(\boldsymbol{q}, \dot{\boldsymbol{q}})$ via GPU-capable PhysX tensor API

2. **Same controller**: Pure-NumPy `ComputedTorqueController` is engine-agnostic — shared by both implementations
3. **PhysX drive fix**: Setting `stiffness=0` and `maxForce=1e4` was critical — the URDF's `effort="10"` blocked the entire torque budget
4. **Trajectory**: SciPy periodic cubic splines with velocity profiling replace Drake's `PiecewisePolynomial`
5. **Cable viz**: Headless Drake `CablePlant` drives USD cylinder prims (eval mode only, not for parallel RL)
6. **GPU-ready**: The loop structure is compatible with IsaacLab's parallelized `ArticulationView` for future RL training

16 Series Elastic Actuator (SEA) Extension

The rigid-tendon CT controller (above) assumes $\boldsymbol{\tau}^{\text{applied}} = \boldsymbol{\tau}^{\text{desired}}$. In the physical system, joint 2 is driven through a compliant cable–spring transmission:

SEA Physical Topology (Joint 2)

$$\text{Motor drum} \xrightarrow{\text{cable}} \textbf{Spring}\ (k_s,\, b_c) \xrightarrow{\text{cable}} \text{Pulley} \rightarrow \text{Link 2}$$

Joint 1 remains rigid direct-drive ($\tau_1^{\text{app}} = \tau_1^{\text{des}}$).

16.1 Motor Rotor Dynamics (Torque Mode)

The motor model (`actuators/motor_dynamics.py`, class `TorqueMotor`) uses 2nd-order rotor dynamics matching the CubeMars AK60-6 ($N = 6$, $\tau_{\text{peak}} = 9$ Nm):

$$\boxed{J_m \ddot{\theta}_m = \frac{\tau_2^{\text{des}}}{N} - b_m\, \dot{\theta}_m - \frac{r_{\text{p}} \cdot F}{N}} \tag{7}$$

Spring extension and cable force:

$$\delta = r_{\text{p}} \left(\frac{\theta_m}{N} - q_2 \right) \tag{8}$$

$$F = k_s \cdot \delta + b_c \cdot \dot{\delta} \tag{9}$$

Unilateral cable model (cables can only pull):

$$T_{\text{green}} = \max(F, 0), \quad T_{\text{red}} = \max(-F, 0), \quad \tau_2^{\text{sea}} = r_{\text{p}} \cdot (T_{\text{green}} - T_{\text{red}})$$

The SEA creates a first-order lag between τ_2^{des} and τ_2^{sea}. For weak springs ($k_s \leq 100$ N/m), this lag causes 20–40 mm tracking errors on the rectangle trajectory — far worse than the ~ 11 mm achieved with rigid tendons.

16.2 SEA Diagnostics

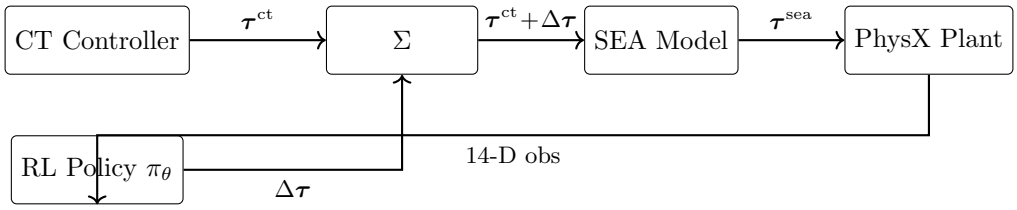
The `SEACableActuatorNP.step()` method returns an `SEADiagnostics` dataclass with ten fields:

Field	Description	Unit
`motor_pos`	θ_m/N (joint-side) or l_m	rad / m
`motor_aux`	$\dot{\theta}_m/N$ or $l_{m,\text{des}}$	rad/s / m
`delta`	Spring extension δ	m
`F_cable`	Net cable force	N
`tau1_des`	Desired τ_1	Nm
`tau2_des`	Desired τ_2	Nm
`T_green`	Retracting cable tension	N
`T_red`	Extending cable tension	N
`tau_sea`	Applied τ_2 via spring	Nm
`tau_motor`	Motor-side electromagnetic torque	Nm

17 Residual RL Compensator

To compensate the SEA lag without hand-tuning a lead compensator, a residual RL policy is trained on top of the CT controller.

17.1 Architecture



The 14-D observation includes joint state, EE error, spring diagnostics (δ, $\dot{\delta}$, F_{cable}), and the CT–SEA torque mismatch ($\tau_2^{\text{des}} - \tau_2^{\text{app}}$). The 2-D action $\Delta\boldsymbol{\tau} \in [-5, +5]^2$ Nm is intentionally small so the RL acts as a
emphcorrection, not a replacement.

17.2 What the Policy Learns for Weak Springs

The analytical solution (what a perfect policy would learn) is a **torque-error I+D compensator**:

$$\Delta\boldsymbol{\tau} \approx k_I \int (\boldsymbol{\tau}^{\text{des}} - \boldsymbol{\tau}^{\text{app}})\, dt + k_D \frac{d}{dt}(\boldsymbol{\tau}^{\text{des}} - \boldsymbol{\tau}^{\text{app}})$$

This inverts the first-order spring lag filter. The RL policy learns this from data without needing to know k_s explicitly, so it **generalizes across spring stiffnesses**.

Specific behaviors for weak springs ($k_s \leq 100$ N/m):

1. **Torque pre-loading**: At trajectory corners, $\Delta\tau_2 > 0$ is sent *before* the corner to pre-charge the spring.
2. **Gravity droop correction**: Steady $\Delta\tau_2 < 0$ on downward segments compensates delayed gravity response.

3. **Active damping**: During spring oscillations, the residual is 180° out of phase.
4. **Cross-joint coupling**: $\Delta\tau_1$ corrects Coriolis errors caused by joint-2's delayed spring response.

See the companion RL notes (`rl/notes/residual_rl_implementation.tex`) for the full reward function, PPO hyperparameters, and training details.

Source files:

- `script_cup_manipulator_pendulam_tendon_computed_torque_isaac_sim.py` — Main CT controller script (project root)
- `script_cup_manipulator_pendulam_tendon_scene_viz_isaac_sim.py` — Scene visualization (project root)
- `robots/cup_manipulator_tendon_isaac.py` — Isaac Sim robot wrapper (URDF→USD, ArticulationView)
- `controller/computed_torque_isaacsim.py` — Pure-NumPy CT controller (engine-agnostic)
- `controller/trajectory.py` — SciPy trajectory classes (Rect, Circle, Line, Preamble)
- `actuators/sea_isaacsim.py` — Pure-NumPy SEA cable model (`SEACableActuatorNP`)
- `actuators/motor_dynamics.py` — Motor dynamics: `TorqueMotor` (2nd-order), `PositionServoMotor` (1st-order)
- `actuators/motor.py` — Motor catalog: AK60-6, AK80-8 configs
- `project_utils/viz_cables_isaacsim.py` — Cable visualization via DrakeCablePlant + USD prims
- `project_utils/log_isaacsim.py` — `IsaacSimLogger`: OS-level fd suppression for quiet startup
- `rl/envs/manipulator_residual_env.py` — Gymnasium environment for residual RL
- `rl/train_ppo_residual.py` — PPO training with SB3 on Isaac Sim
- `rl/eval_ppo_residual.py` — Evaluation: CT-only vs CT+RL comparison plots
- `test_cup_manipulator_tendon_multi_instance_isaac_sim.py` — Multi-instance parallel demo